Software Design Analysis Report

A. Project Details

Project title: AI Research Paper Recommendation System

Team / roll numbers:
Anand Kumar Paswan 202401100100039
Anant Mittal 202401100100041
Anirudh vasishth 202401100100046
Anish Kumar Rai 202401100100047

Project objective (2–3 lines): The system helps students and researchers discover relevant research papers. It analyses a user's search query and interests and recommends papers using their titles, abstracts, keywords, and metadata, returning a ranked list with authors, year, abstract, and source link.

Technology stack: Python for text processing and recommendation logic (TF-IDF / cosine similarity), a web front end (HTML, CSS, JavaScript), a REST backend API, and a relational database for paper metadata and user preferences. [Update to match the technologies actually used.]

B. Module Identification and Modularization

Total number of modules: 6.

The project is divided by responsibility so that interaction with the user, management of paper data, recommendation logic, storage, and administration can be developed, tested, and changed independently. Each module has a focused purpose and communicates with the others only through defined inputs and outputs.

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	User Interface	Accepts queries and preferences; displays recommended papers.	In: query, filters, feedback. Out: ranked papers, messages.	M2, M3, M5
M2	User/Profile Manager	Manages user profiles, interests, saved papers, and feedback.	In: profile data, feedback. Out: preference data.	M4
M3	Paper Search/Metadata Manager	Searches, validates, and retrieves paper metadata.	In: keywords, paper records. Out: candidate papers.	M4
M4	Database Manager	Stores and retrieves users, papers, keywords, and interactions.	In: records, queries. Out: stored/retrieved data.	Database
M5	Recommendation Engine	Extracts text features, computes relevance, and ranks papers.	In: query, profile, papers. Out: ranked list with scores.	M2, M3
M6	Admin/Monitoring	Maintains paper data, manages users, and reports system status.	In: admin actions. Out: updated records, status reports.	M3, M4

Module hierarchy / decomposition diagram:

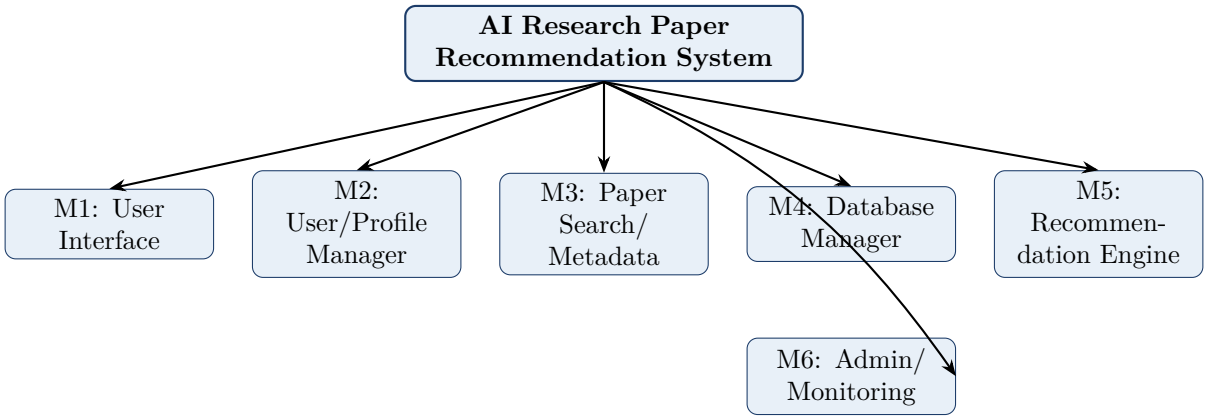


Figure 1: High-level decomposition of the system

C. Design Strategy

Selected strategy: ☐ Top-Down ☐ Bottom-Up ☒ Hybrid

Justification: A hybrid strategy suits this project because the overall goal and the major modules can be defined first from the system level (top-down). Reusable components such as text cleaning, similarity calculation, database access, and ranking can then be built and tested independently (bottom-up). The two are joined through agreed interfaces. This allows team members to work in parallel, keeps planning clear, and makes testing of individual responsibilities easier.

Evidence:

1. Defined the whole system goal: accept a research interest and return relevant papers.
2. Broke the system into six modules (interface, profile, paper data, database, recommendation, admin).
3. Built reusable components: tokenisation, text cleaning, feature extraction, similarity.
4. Connected components through service functions / API interfaces.
5. Integrated the modules and tested the full search-to-recommendation flow.

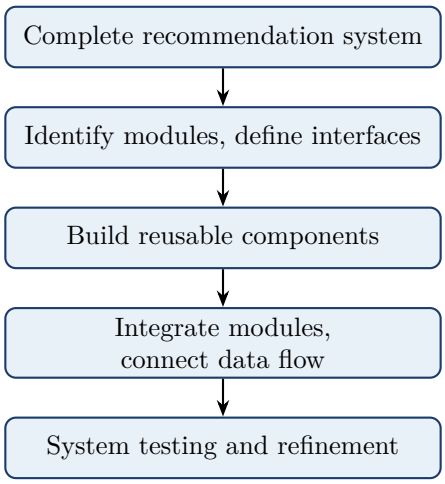


Figure 2: Decomposition and build-up sketch

D. Design Approach

Selected approach: ☐ Function-Oriented ☐ Object-Oriented ☒ Mixed

Justification: The system has clear processing steps as well as real-world entities. Processing steps such as query preprocessing, feature extraction, similarity calculation, and ranking fit a function-oriented view, while users, papers, queries, and recommendations are naturally modelled as objects with attributes and operations. Combining both gives a clear picture of how data is processed and what data is stored.

Main functions: `processQuery()`, `extractFeatures()`, `searchPapers()`, `calculateSimilarity()`, `rankRecommendations()`, `saveFeedback()`.

Main classes / objects: `User`, `ResearchPaper`, `SearchQuery`, `Recommendation`, `RecommendationEngine`.

Evidence – structure chart (function-oriented):

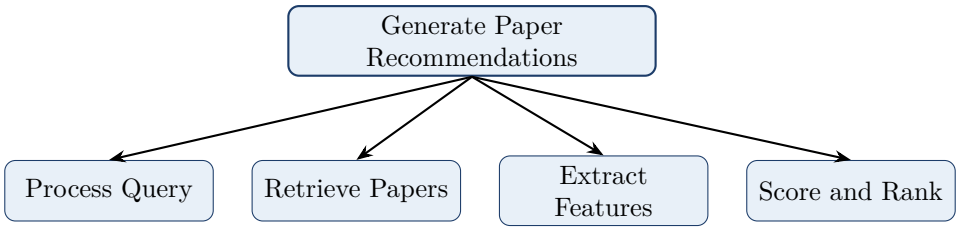


Figure 3: Structure chart of the recommendation process

Explanation: The top function coordinates four sub-functions. The query is first validated and cleaned, candidate papers are retrieved, text features are extracted, and finally each paper is scored and ranked. Results are passed back to the top-level function for display.

E. Coupling Analysis

The table lists important module interactions. The design aims for low coupling by exchanging only the required data through defined interfaces.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M1 – M5	Search query, filters, and ranked results	Data / Low	Use a fixed request/response format; hide internal model details.
M3 – M4	Paper metadata, identifiers, and database queries	Data / Low–moderate	Access data through repository interfaces, not direct queries in other modules.
M2 – M5	User interests, saved papers, and feedback	Data / Low	Pass only the needed preference fields through a service interface.
M5 – M3	Candidate papers and extracted metadata	Data / Moderate	Keep retrieval separate from ranking; exchange standard paper records.
M6 – M4	Administrative updates and status information	Data / Low–moderate	Apply role-based access and controlled database operations.

F. Cohesion Analysis

Each module is evaluated for how closely its internal tasks belong together.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
User Interface (M1)	Accept user input and present search and recommendation results.	Functional / High	Keep recommendation calculations out of the interface.
Paper Search/Metadata Manager (M3)	Retrieve, validate, and organise paper metadata.	Functional / High	Separate external source connectors if their number grows.
Recommendation Engine (M5)	Extract features, calculate relevance, and rank papers.	Functional / High	Split feature extraction and ranking into focused components.
Database Manager (M4)	Store and retrieve user, paper, and feedback records.	Functional / High	Use separate repositories for distinct record types.
User/Profile Manager (M2)	Maintain interests, saved papers, and feedback.	Functional / High	Keep login credentials separate from preference data.

G. Module Interaction and Design Quality

Sequence diagram showing how the major modules exchange information when a user searches for papers:

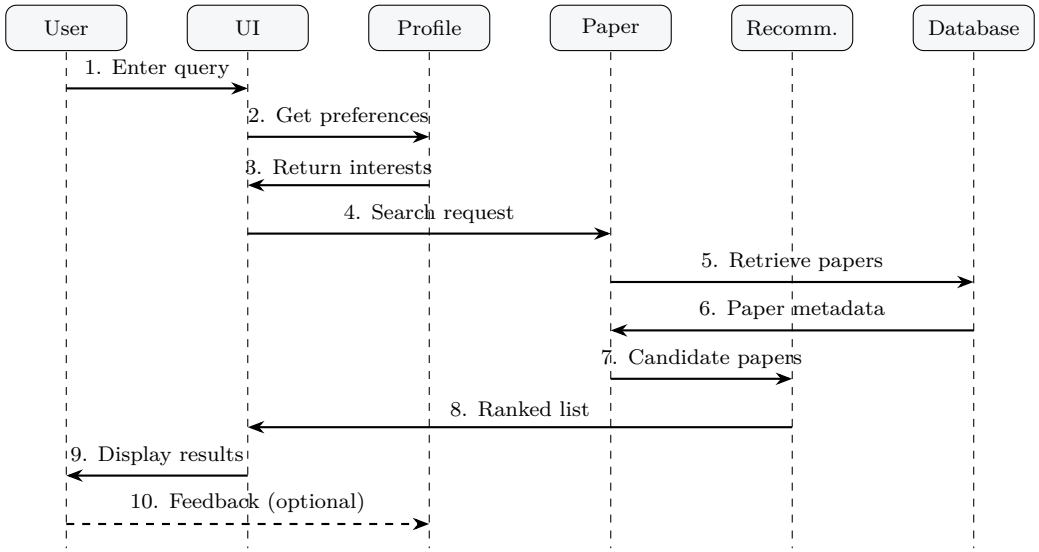


Figure 4: Sequence diagram for a paper recommendation request

Design quality considerations:

- **Modularity:** each module has one distinct responsibility.
- **Maintainability:** the recommendation logic can be improved without changing the interface.
- **Scalability:** paper retrieval and recommendation can be optimised independently.
- **Reliability:** invalid queries and unavailable sources produce clear error messages.
- **Security:** profiles and saved papers are accessible only to authorised users.